SATySFI-formatter

usagrada

目次

1. formatter の install 方法 …… 2

2. formatter の使い方 …… 2

2.1. 終了コード …… 3

2.2. エディタから使う …… 3

3. format option …… 4

3.1. インデント …… 4

3.2. 行幅 …… 4

4. 整形を無効にする …… 4

5. 開発者の方 …… 5

5.1. Pull Request …… 5

5.2. issue …… 5

6. SATySFI-formatter の実装 …… 6

6.1. 構成 …… 6

6.2. 行幅に応じた整形 …… 6

6.3. コメント …… 7

7. document の更新 …… 7

SATYSFI を使うに当たって、formatter が無いのが不便だったので、format をするためのツールを作りました。[*1] [*2]

1. formatter の install 方法

以下のどちらかの方法で、入れることができます。ターミナルに以下のコマンドを打ち込んでください。[*3]

```
cargo install --force --git https://github.com/usagrada/satysfi-formatter.git --branch main
```

```
git clone -b main https://github.com/usagrada/satysfi-formatter.git
cd satysfi-formatter
cargo install --path .
```

2. formatter の使い方

```
satysfi-fmt $input
```

整形結果が標準出力に出ます。ファイルは書き換えません。

```
satysfi-fmt $input --write
```

`-w` または `--write` を指定することで、input で入力したファイルに書き込むことができます。内容が変わらないファイルは書き直しません。

---

1 このドキュメントは format のテストも兼ねて書いています。

2 期待するフォーマットが得られない場合は、Issue や Pull Request いただけると幸いです。

3 --force は無くても入りますが、既にインストールしている場合、最新のデータにアップデートするために同じコマンドを使用できます。

```
satysfi-fmt $input -o $output
```

`-o` または `--output` でファイルを指定して出力できます。

```
satysfi-fmt --check $input
```

`--check` を指定すると、ファイルを書き換えず、整形されていないファイルの名前を標準エラー出力に出します。CI で整形済みかを確認するときに使えます。

```
satysfi-fmt -w *.saty *.satyh
```

複数のファイルは、`--write` または `--check` のときに指定できます。

2.1. 終了コード

- 0: 成功です。`--check` のときは、すべて整形済みです。
- 1: `--check` で、整形されていないファイルがありました。
- 2: エラーです。構文エラー、ファイルの読み書きの失敗、設定ファイルの誤り、引数の誤りが含まれます。

構文エラーのときは何も出力せず、`path:line:column: error: syntax error` の形で標準エラー出力に出します。複数のファイルを渡したとき、途中でエラーになっても残りは処理します。

2.2. エディタから使う

ファイルを指定しない（または `-` を指定する）と標準入力を整形して、標準出力に出します。エディタや language server は、このコマンドを呼べば使えます。

```
satysfi-fmt --assume-filename path/to/main.saty < main.saty
```

`--assume-filename` には、整形する文書のパスを渡します。設定ファイルをそのパスから探すのと、エラー表示に使います。

3. format option

整形の設定は、設定ファイル `.satysfi-fmt.toml` に書きます。コマンドライン引数では変えられません。整形するファイルのディレクトリから親へ向かって探し、最初に見つかったものを使います。`--config` でファイルを直接指定することもできます。

```
line_width = 100       # 行幅 (既定 80)
indent_width = 2       # インデント幅 (既定 4)
use_tabs = false       # インデントにタブを使うか (既定 false)
final_newline = true   # 末尾に改行を入れるか (既定 true)
command_space = true   # コマンドとテキスト引数の間に空白を入れるか (既定 true)
```

すべて省略でき、省略した項目は既定値になります。書き間違い（知らない項目名など）はエラーになります。

3.1. インデント

基本の indent は 4 です。設定ファイルの `indent_width` で、インデントのスペースの数を変えられます。この document は `indent_width = 2` で整形しています。

3.2. 行幅

リストやレコード、関数の引数などは、`line_width` に収まれば 1 行に、収まらなければ縦に展開されます。行幅は目安なので、長い識別子や文字列などは超えることがあります。テキスト（`+p` の中の文章）は、行幅に合わせて折り返しません。

4. 整形を無効にする

手で揃えた表など、整形してほしくない部分は、コメントで整形を止められます。`% satysfi-fmt off` から `% satysfi-fmt on` までの間は、空白や改行も含めて、書いたとおりに出力されます。`on` を書かなければ、ファイルの最後まで整形されません。

```
let x = [1;   2]
% satysfi-fmt off
let table = [
```

```
  [1;   0;   0];
  [0;   1;   0];
]
% satysfi-fmt on
let y = [1;   2]
```

上のコードは、`x` と `y` だけが整形され、`table` は元のままになります。`off` / `on` のコメント自体は、通常どおり字下げが整えられます。

5. 開発者の方

`lib.rs` の `format` を開始地点とし、コードを `satysfi-parser` で CST 化し、`pretty` の Doc に変換して、行幅を指定して描画しています。コメントと空行は、CST の子ノードの間から取り出して Doc に埋め込んでいます。仕様は `docs/spec.md` にまとめています。期待するフォーマットが得られないなど、修正等があれば、Pull Request や Issue にお願いいたします。

Issue でいただく場合、期待するフォーマットのテストをいただけるとスムーズに対応ができると思います。その際、実際にそれがコンパイル可能である必要はありません。

5.1. Pull Request

実装した部分のテストケースを書いていただいてから、プルリクエストをいただけると幸いです。その際、`src/tests` 以下でしたら何処に書いていただいても構いません。

5.2. issue

以下にサンプルを載せておきます。`r#""` の内部に書かれたテキストはスペースや改行を含め全てそのまま出力されるため、スペース数改行数等の違いにより、テストが容易に落ちます。

```
#[test]
fn test1() {
    // format 前のテキスト
    let text = r#"@import: hello
  @require: local
```

```
document(|title = {hello}|)'<+p{hello world}>"#;


    // 期待されるテキスト
    let expect = r#"@import: hello
@require: local


document (|title = { hello }|) '<
    +p { hello world }
>
"#;
    test_tmpl(text, expect);
}
```

6. SATYSFI-formatter の実装

6.1. 構成

`src/printer` 以下に、rule の種類ごとにモジュールを分けています。

- expr: 式、リスト、レコード、関数適用、制御構文
- stmt: `let` や `type`、`module` などの文
- types: 型
- pattern: パターン
- text: 垂直モード、水平モード、数式モード
- directive: `% satysfi-fmt off` / `on` による整形の制御

6.2. 行幅に応じた整形

リストやレコードは、収まれば 1 行、収まらなければ 1 要素ずつ改行します。このとき、縦に展開した場合だけ、最後の要素にも `;` を付けます。演算子の連鎖は、収まらなければ演算子を行頭に置いて並べます。関数適用は、収まらなければ 1 引数ずつ改行します。ただし、最後の引数がリストやレコードなどのときは、括弧を直前の行に置いたまま中身だけを展開します。

6.3. コメント

コメントは消さずに、元の位置に近い場所に出力します。行末のコメントは行末に残し、単独行のコメントは直後の要素の前に置きます。行コメントは行末まで続くので、後ろに続く記号を飲み込まないように、必要なときは次の要素の前の行に移します。

7. document の更新

`draft.saty` を更新して、以下のコマンドを叩くと `doc.pdf` を更新します。

```
cargo make build-doc
```

その際、`cargo-make` というパッケージが必要なので、インストールしていない方は以下のコマンドでインストールしてください。

```
cargo install --force cargo-make
```

`doc` ディレクトリの `.satysfi-fmt.toml` で、インデント幅を 2 にして整形しています。